



FACULTY OF COMPUTING
SEMESTER 2 2025/2026

SECP3843 - SPECIAL TOPIC IN DATA ENGINEERING

SECTION 2

LECTURER: DR. ARYATI BINTI BAKRI

Individual Project

TITLE:

Scalable Data Architecture for Traffic Safety Analytics: Implementing a Medallion Lakehouse and Predictive AI using Apache Spark

NAME: CHOH JING YI

MATRIC NUMBER: A23CS0296

Scalable Data Architecture for Traffic Safety Analytics: Implementing a Medallion Lakehouse and Predictive AI using Apache Spark

Choh Jing Yi

Faculty of Computing
Universiti Teknologi Malaysia
Skudai, Johor, Malaysia
jingyichoh@gmail.com

Abstract — This project develops a scalable data engineering pipeline for analyzing road traffic accidents in the United States by integrating three heterogeneous data sources: the US Accidents dataset, the Nager.Date Public Holiday API, and state population records from the U.S. Census Bureau. The implementation adopts a Medallion Data Lakehouse architecture consisting of bronze, silver, and gold layers using Apache Spark and Delta Lake. Data preprocessing includes timestamp standardization, missing-value handling, feature engineering, and dataset integration to produce analytics-ready data. A Random Forest model implemented with Spark MLlib is incorporated to predict accident severity levels based on temporal, environmental, infrastructural, and demographic factors. The processed data are further organized into a star schema and visualized through interactive Power BI dashboards to support data-driven insights and transportation safety analysis.

Keywords- Data Engineering; Apache Spark; Data Lakehouse; Random Forest; Traffic Accident Analytics; Power BI

I. INTRODUCTION

Road traffic accidents are one of the most public safety problems around the world that cause high numbers of deaths, injuries and economic losses each year. According to the World Health Organization, approximately 1.19 million people die each year caused by road traffic crashes, and 10 out of millions are injured without fatality and result in permanent disability [1]. These incidents cause significant consequences not only for people and families but also for health services, emergency responders, insurance companies and government institutions that are responsible for the safety of transport.

This project focuses on transportation and public safety by analyzing road traffic accident data in the United States. This project integrates three different data sources to build a comprehensive analytical model. The primary dataset is the US Accidents dataset, a public repository from Kaggle containing approximately 7.7 million accident records collected across 49 U.S. states from 2016 to 2023 [2]. Besides that, this project integrates real-time US Public Holiday records fetched via a RESTful API and United States Census Bureau population dataset. These data sources provide a combination of weather conditions, road infrastructure, geographical data, and temporal factors that influence accident severity.

There are several challenges when analysing traffic accident data and predictive modelling when dealing with large dataset. The accident data is collected every day and may cause data quality issues such as missing data, inconsistent formats, duplicate records and different attributes from various sources. These data quality issues must be addressed during data processing otherwise they will bring a large negative consequence on the reliability of analysis outcome and machine learning predictions.

Another challenge is the scalability of data processing. The US Accidents dataset consists of about 7.7 million records and many attributes that describe the characteristics of an accident, weather conditions, road environments and geographical information. Traditional data analysis tool like Pandas is effective for small and medium sized datasets, but it struggles when processing millions of data [3]. Hence distributed computing technologies such as Apache Spark is needed to handle the massive data efficiently. Instead of using one computer, Spark splits data into smaller parts and processes them across multiple computing nodes. This makes large-scale processing faster and more efficient.

While many previous studies have focused on improving accident severity prediction accuracy using machine learning algorithms. The problem is it had limited attention to the development of scalable end-to-end data engineering frameworks that integrate multiple data sources, data storage, transformation analytics, and predictive modelling within a unified architecture. This fragmented data reduces scalability, increases maintenance complexity, and limits the implement predictive analytics solutions. As accident datasets continue to grow and complexity, there is a growing need for solutions that combine both big data processing and machine learning capabilities within a single platform.

The proposed pipeline integrates accident records, public holiday information, and state population statistics into a unified Data Lakehouse environment. The proposed framework categorizes data into Bronze, Silver and Gold as a level for facilitating data ingestion, cleaning, transformation, and

analytical processing. In addition, a machine learning model is developed using Random Forest classification with Apache Spark MLlib to predict the severity of the accident depending on weather, temporal, geographical and road infrastructure factors. The processed gold layer dataset is further integrated with Microsoft Power BI to provide interactive visual analytics that support traffic safety monitoring and data-driven decision making.

This project aims to achieve three main objectives. First, develop a scalable Medallion Data Lakehouse pipeline using Apache Spark that able to ingest and process the US Accidents dataset with two supporting sources within a medallion architecture. This helps to ensure systematic data management, efficient processing of large-scale datasets and better data traceability. Second, enhance the data quality by conducting data cleaning and feature engineering. The silver layer preprocesses data to handle missing values, inconsistencies and standardise data format. Moreover, features relevant to the data are designed to improve the analytical quality of the data. These processes can contribute to a reliable and high-quality data set which can be used for accurate analysis and machine learning applications. The last goal of this project is to build and evaluate an accident severity prediction model. It uses Apache Spark MLlib to train and evaluate a machine learning classification model such as Random Forest. The model will use weather conditions, temporal information, and location-based features to predict accident severity in supporting traffic safety analysis effectively.

II. LITERATURE REVIEW

A. Evolution of Scalable Data Architectures

A data pipeline is a structured process that collect, transform, and ingest data from various sources to a data storage and analytic process. Modern organizations rely on data pipelines to automate movement of data between different systems or data sources to ensure data quality, consistency, and scalability. Data pipelines are the foundation of modern data engineering systems by ingesting, processing, storing and analysing large, growing, and complex datasets reliably. As the volume and variety of data increase, scalable architectures become essential for supporting efficient data pipeline implementation.

Modern information systems generate massive volumes of data, bringing significant challenges for traditional data management platforms. While traditional relational databases are primarily designed for structured and moderate size datasets, they face significant limitations when handling large datasets that require complex analytical computations. There are increasing amounts of information from web, mobile, email, social media, and networked smart devices. It is important to have a scalable and efficient data architecture. Traditional data processing is disk-based and uses RAM to process data creates major performance bottlenecks when handling massive and modern datasets [4]. These limitations become more obvious

when datasets contain millions of records that require transformation, aggregation and analytical processing. Distributed computing frameworks have emerged as major parts of modern data architectures to tackle these challenges. For example, Apache Spark is mainly used for efficient large-scale data analytics. Apache Spark uses in-memory computing, where data is processed in memory and not repeatedly read from disk storage. This method can process data up to 100 times faster in memory and 10 times faster on disk than traditional frameworks like Hadoop MapReduce [5]. In addition, Apache Spark offers a single platform for data engineering, machine learning, stream processing, and SQL analytics in a single environment.

As data processing continues to evolve, organizations have begun to adopt more flexible data storage architectures. Data warehouses offer a rigid schema and easy to maintain data governance, but it is difficult to manage various data formats. In contrast, data lakes provide scalable storage for structured, semi-structured and unstructured data but it may cause the phenomenon commonly referred to as a data swamp when lack of data governance and poor data quality [6]. The Data Lakehouse architecture addresses the limitations of both data warehouses and data lakes. Data Lakehouse was introduced as a hybrid approach that combines the governance and performance capabilities of data warehouses with the scalability and flexibility of data lakes. The Lakehouse architecture combines the flexible data storage of a lake and the high-performance analytics capabilities of a warehouse into one solution.

The Medallion Architecture is among the most popular architectures that implement the Lakehouse concept. The Medallion Architecture is a framework that organize data into multiple layers, bronze, silver and gold. The bronze layer contains raw data from source systems, the silver layer is for cleaning and standardizing the data while gold layer has business-ready data optimized for reporting, analysis, and machine learning. This helps to manage data quality, simplified maintenance, and better scalability by separating different stages of data processing.



Figure 2.1 Medallion Data Lakehouse Architecture [7]

As shown in Figure 2.1, data moves from the bronze layer, where raw data are stored, to the silver layer for cleansing and standardization. The gold layer is analytics-ready datasets that can be used for analytics and machine learning purposes. This multi-layered architecture allows to support data ingestion,

cleansing, feature engineering, and preparation for machine learning while maintaining data quality and scalability throughout the pipeline.

B. Distributed Machine Learning Using Apache Spark

Modern information systems generate massive data that brings challenges of computational scalability and hardware limitations. Traditional machine learning is built on single node architecture that will suffer from memory constraints and computational overhead when dealing with large datasets. While Apache Spark solve the challenges by distributing both data and computation across multiple worker nodes. This parallel processing capability allows large datasets to be processed simultaneously, significantly reducing training time and improving scalability. Spark's in-memory processing model minimizes disk I/O operations and enables efficient iterative machine learning computations. As datasets continue to grow in term of size and complexity, distributed machine learning has emerged as an effective approach for improving scalability, reducing processing time, and supporting large-scale predictive analytics.

Apache Spark enables machine learning workloads to be executed efficiently within a distributed computing environment. Spark has a built-in library of machine learning algorithms called Spark MLlib. It provides a wide range of algorithms for classification, regression, clustering and feature engineering algorithms. MLlib provides efficient functionality for a wide range of learning settings and includes several underlying statistical, optimization, and linear algebra primitives. Rather than forcing a single computer to memorize millions of rows at once, it divides the highly structured data into smaller and manageable partitions and distributes them across a cluster of interconnected nodes [8].

One of the key advantages of Spark MLlib can integrate data processing and machine learning within a unified framework. In previous analytic workflows, data preparation and model development are often performed using separate tools. It creates additional complexity and data movements between systems. Spark allow data ingestion, transformation, feature engineering and model training conduct in the same environment to improve efficiency and simplifies the development of end-to-end analytics pipelines.



Figure 2.2: Spark MLlib Workflow

The Spark MLlib workflow has 5 major steps as shown in Figure 2.2. It comprises data preparation, feature engineering, training, evaluation and deployment. The process begins with raw data collected from sources, then it is processed, and do data cleaning activities to enhance the data quality. The next step is feature engineering. It involves transform raw attributes into machine-learning features. After feature preparation, the processed data will be used to train machine learning models.

Spark MLlib includes many distributed learning algorithms, enabling the efficient training of a model using multiple computing resources. Once trained, the model is evaluated for its predictive ability using the suitable performance metrics. These models that achieve satisfactory results will be deploy for use in decision making.

Among various classification, Random Forest is suitable for transportation datasets because it can handle high-dimensional data, nonlinear relationships, and interactions among multiple variables without requiring strict statistical assumptions. Random Forest is an ensemble learning algorithm that builds multiple decision trees and pools the results of the trees to make a single prediction. The algorithm can detect the complex nonlinear relationship between variables and can process the data set with both numerical and categorical variables. These characteristics make Random Forest approach suitable for accident severity prediction. In this project, the prediction model utilizes weather conditions, temporal variables, road infrastructure characteristics, public holiday information, and state population data. These integrated variables create complex relationships that are difficult to capture using conventional statistical methods. Therefore, Random Forest provides a suitable approach due to its ability to model nonlinear interactions among multiple features.

Distributed machine learning framework like Apache Spark MLlib provides an effective platform for developing predictive analytics solutions on large-scale datasets. By integrating data processing, feature engineering, model training, and evaluation within a unified environment, Spark supports the development of scalable end-to-end machine learning pipelines. These capabilities make Spark MLlib effective in accident severity prediction using the large-scale dataset in this project.

C. Accident Severity Prediction in Transportation Analytics

Accident severity prediction helps traffic analyst to understand the risks of accidents at certain places and environment to plan road safety more effectively. Accurate prediction of accident severity supports emergency response optimization, identification of high-risk locations, and the development of targeted road safety interventions.

Previous studies have shown that accident severity is influenced by multiple related factors such as road environment, external conditions and driver characteristics. Eboli, Forciniti, and Mazzulla found that road geometry, traffic conditions, weather, and driver-related factors all significantly contribute to crash severity levels [9]. Similarly, research by Yu and Abdel-Aty found that real-time weather and traffic variables, including temperature and seasonal conditions play a significant role in determining accident outcomes [10]. These findings indicate that severity of an accident is multiple factors and cannot be explained by a single cause. Thus, it is a well-suited issue for analysis through data-driven and machine learning techniques.

These factors are well represented in the US Accidents dataset, which contains attributes such as weather conditions, temporal characteristics, geographical locations, and road infrastructure attributes. These attributes help developing machine learning models that can identify patterns related to accident severity and support traffic safety analysis. In addition, this project uses external datasets such as US total population by state and public holiday information to improve predictive accuracy. Area with higher population normally experience greater vehicle density, increased road usage, and a higher probability of accident rate. External factors such as public holidays also involve traffic behavior patterns. L.Qiu and W. A. Nixonshows found that traffic volume and driving behavior change significantly during holiday periods due to increased travel demand and longer distance trips. These changes often lead to variations in accident frequency and severity [11]. This integration enables a more comprehensive representation of real-world driving conditions, supporting more accurate accident severity prediction.

As the availability of transportation data continues to increase, researchers use machine learning algorithm to improve the accuracy of accident severity prediction. Among various classification algorithms, Random Forest is the most widely adopted algorithms for accident severity classification due to its ability to handle large datasets model complex relationships between variables and produce reliable prediction results. Random Forest is an ensemble machine learning algorithm that builds multiple decision trees and combines their predictions to improve accuracy and reduce overfitting [12]. Furthermore, Apache Spark MLlib allow large-scale model training and evaluation on millions of accident records to implement efficiently. These characteristics make it a suitable approach for accident severity prediction using the US Accidents dataset.

Although machine learning algorithms can successfully predict accident severity. However, most research primarily focuses on improving predictive accuracy and provide less resource on data engineering infrastructure that required to ingest, clean, transform, and manage large-scale accident datasets before model development. This limitation highlights an important research gap that motivates the development of the proposed framework in this literature review.

D. Research Gap

This literature shows a strong relationship between scalable data processing architecture, distributed machine learning framework and accident severity prediction. Apache Spark is widely adopted for big data analytics due to its distributed computing and in-memory processing making it suitable for handling large-scale accident datasets. Data Lakehouse architecture is widely used for storing structured and unstructured data management and improve data quality and governance at the same time. Machine learning algorithms such as Random Forest are effective in predicting accident severity.

However, most existing research on accident severity prediction mostly focuses on enhancing the accuracy of the model and comparing machine learning algorithms. Although these studies provide valuable insights on the accuracy of the predictions, they often overlook the underlying data engineering processes required to support large-scale analytics, including data ingestion, cleaning, transformation, and feature engineering within a unified pipeline.

On the other hand, research on Apache Spark and Data Lakehouse architectures focuses primarily on enhancing data storage, processing power and scalability. Very few studies provide an example of how these technologies can be combined with machine learning models in a single framework to support transportation safety analytics.

Thus, there is a research gap in developing a framework that integrates scalable data engineering practices with distributed machine learning for accident severity prediction. This project will process an end-to-end Medallion architecture using Apache Spark, Data Lakehouse and Random Forest classification in a single analytical tool. The framework will structure the data in bronze, silver and gold layer. This integrated framework helps to process data efficiently, improve data quality and predict accident severity accurately.

III. METHODOLOGY

A. Methodology

The Waterfall methodology was selected as the primary software development life cycle (SDLC) model for designing and building this scalable data engineering pipeline and predictive machine learning model. The Waterfall model is a sequential and structured development methodology where each phase must be completed before moving to the next stage. The key steps involved in this project are data requirement analysis, data cleaning, data ingestion, data processing, machine learning modelling and data visualization. The Waterfall approach was chosen because this is one-time end-to-end data engineering pipeline project. Therefore, Waterfall can be used for controlling and organizing the process of completion of every stage of the data engineering pipeline before moving on to machine learning and evaluation phases.

B. Data Sources Identification

This project integrates three heterogeneous data sources consisting of one primary fact dataset and two supporting dimension datasets. These datasets are initially independent and are later integrated within the Medallion Data Lakehouse architecture. Table I shows the role of each source.

TABLE I. DATA SOURCES

Dataset	Type	Format	Source	Purpose
US Accidents	Fact	CSV	Kaggle	Primary accident records
US Public Holidays	Dimension	JSON/API	Nager.Date API	Holiday feature engineering

Dataset	Type	Format	Source	Purpose
US State Population	Dimension	CSV	US Census Bureau	Population normalization

US Accidents dataset is the primary dataset retrieved from Kaggle website. It contains approximately 7.7 million traffic accident records across the United States from 2016 to 2023. This dataset includes attributes such as accident severity, start and end time, geographic coordinates, weather conditions, and road infrastructure features such as junctions and traffic signals.

The second dataset is retrieved from the Nager.Date Public Holiday API which provides structured JSON data that store United States public holidays. This dataset includes holiday dates and names to better identify whether an accident occurred on a public holiday. The third dataset is retrieved from the United States Census Bureau, which provides total population data for all states. This dataset includes state name and total population per state. This dataset is used as a demographic dimension table to support normalization of accident frequency. The population data allows to analyse more meaningful indicators such as accident rate per 100,000 population. This ensures fair comparison across states by reducing bias toward highly populated regions.

C. Data Ingestion (Bronze Layer)

The bronze layer is the initial stage of the data pipeline. This layer collected raw data and stored in its original form. In this project, PySpark is used as the main tool for data ingestion because it can handle large-scale and heterogeneous data efficiently. Besides that, it supports distributed processing, which is useful when dealing with millions of records like in the case of the US Accidents dataset.

There are three datasets: US accident CSV, public holidays JSON via API, and US state population CSV are ingested independently into its original formats. After three datasets are successfully ingested, they are stored in Delta tables to maintain their original structure and data lineage for downstream processing. No changes of missing or incomplete values at this stage because the bronze layer prioritizes raw data preservation.

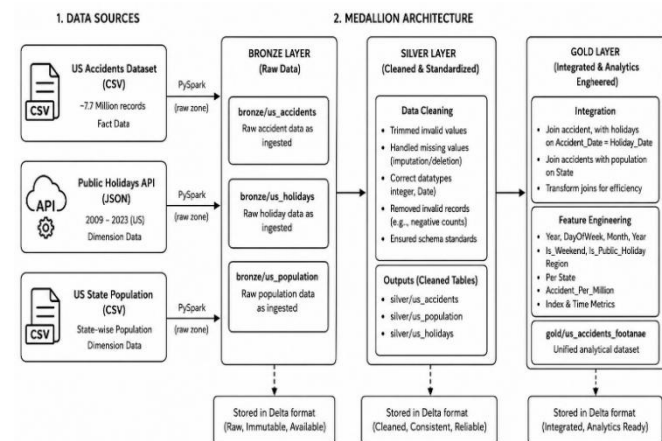


Figure 3.1: Data Integration Plan for Medallion Pipeline

Figure 3.1 shows bronze-to-gold ingestion and integration flow. After data ingestion, silver layer subsequently performs data cleaning, standardization, and type conversions, while the gold layer integrates the datasets through common attributes such as state codes and accident dates. Additional features, including holiday indicators, temporal variables, and population information, are generated to support both predictive modelling and business intelligence applications.

TABLE II. COMMON SCHEMA MAPPING

Data sources	Key Attributes	Target Table	Roles
US Accidents (CSV)	State, Start_Time, Severity	Fact_Accidents	Core accident events
Public Holidays API (JSON)	date, name	Dim_Holidays	Temporal dimension
State Population (CSV)	State_Name, Population	Dim_Population	Demographic dimension

Table II shows how the three data sources are mapped into a unified analytical structure. Missing or incomplete values are preserved at the bronze while data processing and feature engineering are conducted in following silver and gold layers.

D. Data Processing and Transformation (Silver Layer)

The silver layer is responsible for conducting data cleaning, standardization, and validation, using PySpark DataFrame operations to process the dataset efficiently. The activities include converting timestamps into a standardized datetime format to enable accurate calculation of accident duration. Second, missing values related to weather such as temperature, humidity, wind speed, precipitation, and visibility are handled through replace with null values or domain-specific values while data with missing critical fields such as timestamps or geographical coordinates are removed. Third, boolean data types attributes such as traffic signals, crossings, and junction indicators, are converted into binary numerical values to ensure compatibility with statistical analysis and predictive modelling. Lastly, duplicate records in the dimension datasets are also removed to prevent redundant results in analytical and reporting stages. Table III shows the cleansing activities in the silver layer.

TABLE III. DATA CLEANSING ACTIVITIES IN THE SILVER LAYER

Processing Activity	Implementation	Purpose
Timestamp standardization	Convert Start_Time and End_Time to datetime format	Ensure temporal consistency and enable duration calculation
Duration calculation	Compute Duration_minutes from accident start and end times	Create an additional analytical feature
Missing weather values	Replace null values with default or domain-specific values	Preserve data completeness and reduce information loss

Critical null removal	Remove records with missing timestamps or coordinates	Maintain analytical reliability
Boolean conversion	Convert infrastructure attributes to binary values (0/1)	Support machine learning algorithms
State code standardization	Map full state names to two-letter abbreviations	Enable dataset integration across sources
Duplicate handling	Apply distinct records within dimension tables	Prevent redundant analytical results

E. Feature Engineering and Integration (Gold Layer)

The gold layer processes and transform the cleaned datasets into an integrated analytics model that supports both machine learning and business intelligence applications. In this layer, US Accidents dataset acts as the central fact table, while the public holiday and population datasets are the dimension table by joining accident data with holiday and population data using accident dates and state. Besides that, several attributes are generated such as temporal, holiday based, and demographic characteristics to investigate the relationship to accident severity. Table IV shows the major feature engineering activities in the gold layer.

TABLE IV. FEATURE ENGINEERING ACTIVITIES IN GOLD LAYER

Processing Activity	Implementation	Purpose
Temporal feature extraction	Derive Hour, Month, Year, and DayOfWeek from Start Time	Support time-based traffic pattern analysis
Weekend and rush-hour indicators	Generate Is_Weekend and Is_Rush_Hour from temporal attributes	Identify high-risk traffic periods and commuting behaviour
Night-time indicator	Generate Is_Night based on accident occurrence time	Capture visibility-related risk factors
Holiday feature generation	Match accident dates with public holiday records and create holiday indicators	Analyse the influence of public holidays on accident occurrence and severity
Population integration	Join accident records with state population data using standardized state codes	Support demographic analysis and population-normalized metrics
Distance-per-minute calculation	Compute Distance_Per_Minute from Distance_mi and Duration minutes	Provide an additional indicator for analysing accident movement characteristics

The gold dataset then fed into Random Forest classifier and transformed into a multidimensional structure which consist of one fact table and two-dimension tables to develop an interactive dashboard.

F. Data Storage

The clean datasets are stored using a Data Lakehouse architecture implemented with Apache Spark and Delta Lake. A Data Lakehouse combines the benefits of the flexibility of traditional data lakes and data governance of data warehouses. This storage method helps to manage structured and semi-structured data in a unified environment while supporting large-scale analytical workloads.

In this project, the storage layer is designed using a medallion architecture which consist of bronze, silver, and gold layers. The bronze layer preserves raw datasets in their original form to ensure data lineage and reproducibility. The silver layer stores cleaned and standardized datasets after preprocessing operations, while the gold layer contains integrated and feature-engineered datasets that are ready for machine learning and business intelligence applications. All layers are store as Delta tables, while the final gold data are exported as CSV files for Power BI visualization.

TABLE V. DATA LAKEHOUSE STORAGE STRUCTURE

Layer	Stored Data	Storage Format	Purpose
Bronze	Raw accident, holiday, and population datasets	Delta	Preserve original data and maintain data lineage
Silver	Cleaned and standardized datasets	Delta	Ensure data quality and consistency
Gold	Integrated analytical dataset and star schema tables	Delta (internal), CSV (Power BI export)	Support machine learning and Power BI analytics

The gold layer is then exported into a star schema consisting of Fact_Accidents, Dim_Holidays, and Dim_Population tables for Power BI integration. Star schema design improves query speed, simplifies dashboard development, and provides a scalable foundation in data analytics.

G. Artificial Intelligence (AI) algorithm

Random Forest classifier is implemented in the gold layer using Apache Spark MLlib to predict accident severity. The model incorporates 21 variables spanning spatial, temporal, environmental, infrastructure, holiday, and demographic dimensions. The dataset was split into 80% training and 20% testing subsets using randomSplit() with a fixed random seed to ensure same results produced. Random Forest was chosen over simpler models such as logistic regression because it can capture nonlinear interactions among these diverse feature types without making assumptions towards the underlying data.

TABLE VI. MODEL CONFIGURATION AND EVALUATION RESULTS

Parameter / Metric	Value
Number of trees	50
Maximum tree depth	9
Train/test split	80% / 20%
Accuracy	81.27%
Weighted F1-score	0.7567
Weighted Precision	0.7946
Weighted Recall	0.8127

The accuracy of the Random Forest model was 81.27%, showing that it classified most number of accidents correctly. The weighted F1-score (0.7567) was slightly lower than the accuracy value, which suggests that the model performed

differently across the four severity categories. This result is because the dataset is heavily dominated by severity level 2. In addition, the weighted precision (0.7946) and recall (0.8127) show that the model was generally accurate and able to identify majority of accident cases. Overall, the findings demonstrate that the proposed approach is effective for accident severity prediction.

IV. SYSTEM DESIGN & SYSTEM ARCHITECTURE

A. System Design

This project's system aims to design an end-to-end data engineering and analytics pipeline that transform raw accident data into useful insights. The system is built on a Medallion Data Lakehouse architecture and implemented using Apache Spark and Delta Lake. The architecture is structured into three layers, bronze, silver and gold layer. Then gold layer branches into Random Forest classifier for predicting accident severity and Power BI for interactive dashboard reporting. The overall system workflow is summarised as Figure 5.1 below:

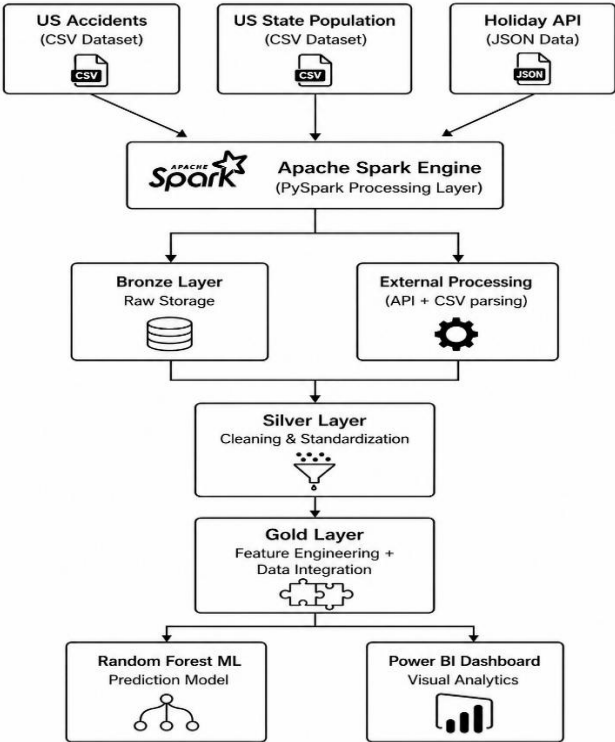


Figure 5.1: High-Level System Architecture

B. Use Case Diagram

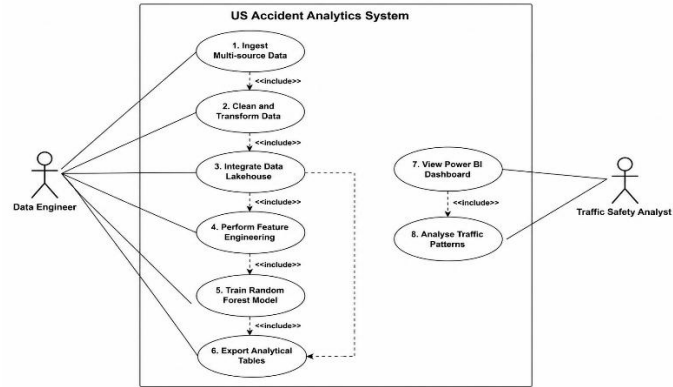


Figure 5.2: Use Case Diagram

Use case diagram is proposed to implement the road traffic accident analytics system as shown in figure 5.2. Two primary roles are involved in this system which is data engineer and traffic safety analyst.

The Data Engineer is responsible for the technical aspects of the data pipeline implementation and management. This involves ingesting data from various sources. Then, the system do automated data preprocessing such as data cleaning, standardization, handle missing value and feature engineering within the Medallion architecture. Once the datasets are integrated in the Data Lakehouse, the Data Engineer trains the Random Forest model for accident severity prediction and evaluates model performance. Lastly, the output data is structured as a star schema and presented in power BI interactive dashboards.

The second roles are the traffic safety analyst who uses Power BI to analyse the accident patterns to make informed decision making. The analyst can analyse factors that may influence road safety. The analyst also reviews predictions to better understand factors that contribute to severe accidents. These insights help to support better traffic planning and resource allocation. The proposed system allows both data management and decision support activities to perform efficiently in a single integrated platform.

C. Architecture Diagram

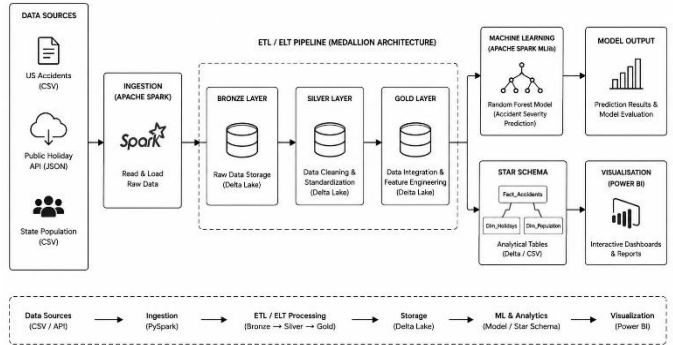


Figure 5.3: Architecture Diagram

The system architecture used in this project is described in figure 5.3. The process starts with three data sources: the US Accidents dataset in CSV format, the Nager.Date Public Holiday API in JSON format, and the US State Population dataset obtained from the Census Bureau. Then these data sources are ingested into the bronze layer without changing their original structures using Apache Spark.

After that, data pre-processing tasks are conducted in silver layer. It involves standardising timestamp format, handling the missing values, removing the invalid records and converting traffic signals and junctions into numerical values. These operations enhance data quality and data consistency in various datasets.

After cleaning, the gold layer connects accident records with holiday and population information using accident dates and state. The pipeline generates a feature-engineered dataset for machine learning and a star schema consisting of Fact_Accidents, Dim_Holidays, and Dim_Population for Power BI analysis. New attributes such as rush-hour indicators, weekend flags, holiday status, and temporal attributes, are also created to support analytical tasks. All processed data are stored in Delta Lake to maintain high durability and schema consistency.

There are two analytical outputs are produced from the gold layer. First, a Random Forest model implemented using Apache Spark MLlib predicts accident severity and generates performance metrics such as accuracy, precision, recall, and F1-score. Second, the integrated data are exported into a star schema table for visualization in Power BI. This architecture creates a complete pipeline that connects data ingestion, storage, artificial intelligence, and BI reporting in a single and scalable framework.

D. Data Pipeline Implementation

The data pipeline is implemented using PySpark within a Google Colab environment and follows a sequential Medallion workflow. The implementation starts by reading three independent data sources: the US Accidents CSV file loaded using `spark.read.csv()` with `inferSchema` enabled to automatically detect data types, the Nager.Date Public Holiday API loaded using `spark.read.json()` to parse the nested JSON response into a structured `DataFrame`, and the US State Population CSV file. Each source is first stored as a Delta table in the bronze layer to preserve the original data and maintain reproducibility.

The subsequent processing is performed in the silver layer through Spark `DataFrame` transformations. It involves standardized timestamp format, handling the missing values, and removing the invalid records. Infrastructure attributes such as traffic signals, crossings, and junctions are also transformed into binary numerical values to enable analytical processing.

The gold layer joins accident records with holiday and population datasets using accident dates and state. Then, feature

engineering is conducted to derive temporal indicators such as hour, month, and day of the week, generate holiday and rush-hour flags to support machine learning modelling and business intelligence reporting. Finally, the processed data are exported into a star schema structure consisting of one fact table and two-dimension tables to support analytical queries.

E. Database Design

The database design contains two perspectives, conceptual entity-relationship model that describes the original data sources and a multidimensional star schema that supports analytical processing and dashboard reporting. The conceptual model reflects the relationship between accident records, public holidays, and state population information. Figure 5.4 shows the conceptual ERD of the source datasets. The US Accidents dataset is the primary entity and joining with public holidays through accident dates and with demographic information through state attributes.

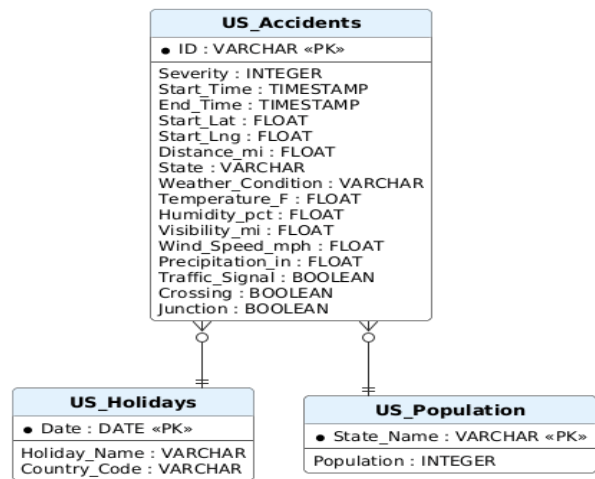


Figure 5.4: Conceptual Entity-Relationship Diagram of Source Data

After preprocessing and feature engineering, the data are transformed into a multidimensional database model. The analytical structure follows a star schema consisting of a central fact table, Fact_Accidents, linked to two-dimension tables, Dim_Holidays and Dim_Population is shown in figure 5.5. The star scheme was selected for this project because it has fewer joins and faster aggregation speed that helps to improve Power BI reporting performance. Some measures are also derived in the star such as total accidents, average severity, and accidents per 100000 population. This database structure provides an efficient foundation for interactive dashboards and predictive modelling tasks.

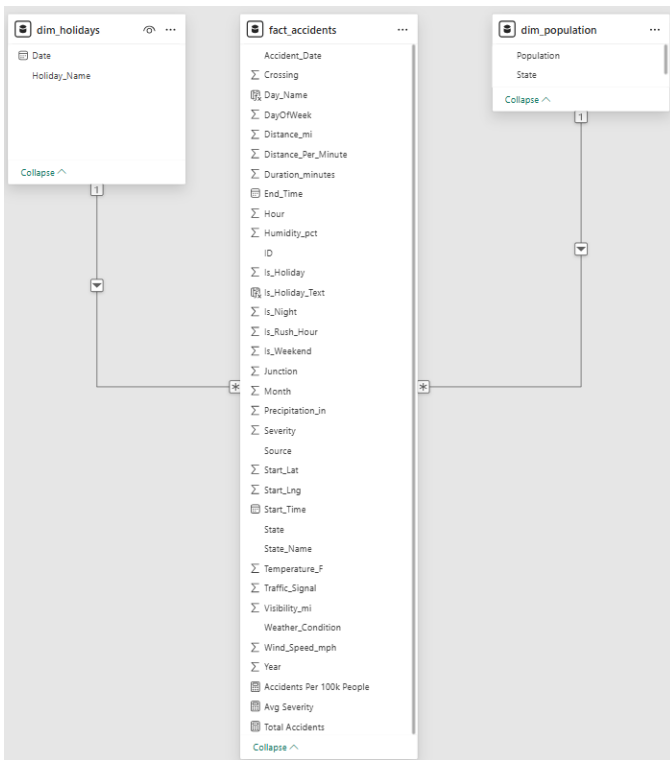


Figure 5.5: Multidimensional Database Design

F. Main Interface



Figure 5.6: Main Interface of the Traffic Accident Analytics Dashboard

The main interface of this system is developed using Microsoft Power BI, and it acts as the dashboard for traffic analytic to analyse accident trends and machine learning results. The dashboard layout includes multiple key performance indicators (KPIs), slicers and visual charts to better make data-driven decision support rather than guesswork.

At the upper section of the dashboard shows three KPI cards: total accidents, average severity and accidents per 100000 people. These cards let users immediately have an overview of national traffic conditions and accident patterns.

The middle section contains slicer that allow users to select states and compare accident behaviour across different geographical regions. Once a state is selected, all charts update

dynamically to allow for analysis of accident frequency, weather conditions, and temporal trends. Besides that, it includes an "Is It Holiday" filter that allows for compare accident patterns during public holidays against regular days

The bottom section of the dashboard provides several visualizations. There are charts showing accident counts by weekday, yearly accident trends, weather-condition distributions, and average severity by hour. Each visualisation charts help analyst identify high-risk periods and environment influences to avoid accident occur in future.

Overall, this dashboard works as an integrated business intelligence interface that transforms large-scale accident datasets into actionable insights. It combines interactive filter, KPIs, and analytical visualizations to have a better long-term planning of road safety.

G. AI Algorithm

The proposed system includes an Artificial Intelligence (AI) algorithm to improve traffic accident analysis based on historical accident records and feature-engineered variables. Random Forest Classifier is used for machine learning in this project after the gold layer of the data pipeline. Random Forest was selected because it is a stable and reliable machine learning method for classification problems. It performs well with large datasets and does not easily overfit when compared to a single decision tree. It is also suitable for mixed data types, which is important in this project since the dataset contains numerical, categorical, spatial, and temporal attributes.

The target variable for AI model is the Severity level (1–4). The input features consist of the gold dataset that include accident location, weather conditions, time-based attributes, road infrastructure indicators, and demographic information like population. These features allow the model to learn patterns of when and where severe accidents are more likely to occur.

Before training, all selected features are converted into a single feature vector using the VectorAssembler component in Apache Spark MLlib. This step ensures that the dataset is in a proper format for machine learning processing. The dataset was then split into 80% training and 20% testing subsets using randomSplit() to evaluate model performance fairly.

The Random Forest model produces prediction outputs for accident severity, which are then compared against actual values. These predictions are used to generate evaluation metrics such as accuracy, precision, recall, and F1-score. Then, a confusion matrix is produced to visualize classification performance in more detail. The final outputs of the model are integrated into the Power BI dashboard. It allows users to interpret historical accident patterns with the model's severity predictions exported from the gold layer.

H. Testing

Once the development process was completed, the proposed road traffic accident analytics system was tested to verify its functionality and ensure that it satisfied the predefined system requirements. This project emphasizes alpha testing as the primary evaluation mechanism. Alpha testing is conducted to determine whether all system features work as intended, whether the implemented modules fulfil the specified requirements, and whether the overall data pipeline functions reliably within the development environment.

The testing activities covered all major components of the system, including data ingestion, data processing, feature engineering, machine learning, data visualization, and dashboard interaction. Each function was executed and validated using real project data. The results show that the proposed pipeline successfully integrates heterogeneous data sources, performs large-scale preprocessing using Apache Spark, generates analytical features within the Medallion architecture, and supports both predictive modelling and business intelligence applications. Table VII shows the results of the alpha testing.

TABLE VII. ALPHA TESTING RESULTS

No	Function	Actual Result	Result	
			Accept	Reject
1	Data Ingestion	The system successfully ingests accident, holiday, and population data into the Bronze layer without data loss.	Yes	
2	Data Cleaning	Missing values, incorrect formats, and invalid records are handled according to predefined rules.	Yes	
3	Feature Engineering	Temporal, holiday, demographic, and traffic-related features are generated correctly.	Yes	
4	Data Integration	The system successfully joins fact and dimension datasets using state codes and accident dates.	Yes	
5	Data Storage	Bronze, Silver, and Gold datasets are stored in Delta Lake format with preserved data lineage.	Yes	
6	Machine Learning Model	The Random Forest classifier trains successfully and produces prediction outputs and evaluation metrics.	Yes	
7	Dashboard Functionality	Power BI visualizations, filters, and interactive elements operate correctly.	Yes	
8	System Objectives	The proposed analytics system fulfils the project requirements for accident analysis, prediction, and decision support.	Yes	

Table VII shows all functional components successfully passed the alpha testing process. Each module works as intended and meet specified requirements. The integration of

Apache Spark, Delta Lake, Random Forest classification, and Power BI provides a complete analytical workflow that supports large-scale traffic accident analysis. The successful completion of alpha testing in this project provides confidence that the system is sufficiently reliable for academic evaluation and future enhancement.

V. CONCLUSION

This project shows a scalable data engineering pipeline for traffic safety analytics by integrating multiple data sources within a unified Data Lakehouse environment. The proposed system integrates the US Accidents dataset with public holiday date and state population data to analyze road safety and accident severity

This project implements a Medallion Data Lakehouse architecture consisting of bronze, silver, and gold layers. Raw data are preserved in the bronze layer, cleaned and standardized in the silver layer, and prepared for analytics in the gold layer. This multi-layered approach maintains data lineage, enhances data quality and facilitates effective feature engineering for downstream applications. Apache Spark enables efficient processing of millions of records while Delta Lake offers reliable storage with schema consistency for large analytical workloads.

Machine learning is incorporated through a Random Forest classifier that predicts accident severity levels using temporal, environmental, infrastructure, and demographic features. The evaluation metrics such as accuracy, precision, recall, and F1-score, suggest that the approach can identify meaningful patterns in the historical accident data.

The Power BI dashboard transforms the processed data into interactive visualizations, enabling exploration and decision-making. Users can examine the trends of accidents by state, investigate weather conditions, determine high-risk periods, and compare the trends of holiday and non-holiday periods. A star schema also provides better efficiency in queries and helps build dashboards.

The objectives of this project have been achieved. Multiple heterogeneous data sources were successfully integrated, a full Data Lakehouse pipeline was implemented, and an Artificial Intelligence algorithm was added to the analytical pipeline. The system successfully shows data engineering methods can help in transportation analytics and provide useful insights for future road safety planning and research.

REFERENCES

- [1] World Health Organization, "Road traffic injuries," *World Health Organization*, 2023. <https://www.who.int/news-room/fact-sheets/detail/road-traffic-injuries>
- [2] S. Rao Nelluri, "Enhancing Data Processing Efficiency Using Apache Spark: Techniques and Optimization Strategies," *International Journal for Research Trends and Innovation*, vol. 7, no. 9, 2022, doi: <https://doi.org/10.56975/ijrti.v7i9.207192>.

- [3] S. Moosavi, "US Accidents (2016 - 2023)," *Doi.org*, 2016. <https://doi.org/10.34740/KAGGLE/DS/199387>
- [4] V. Gurusamy, S. Kannan, and K. Nandhini, "The Real Time Big Data Processing Framework Advantages and Limitations," *International Journal of Computer Sciences and Engineering*, vol. 5, no. 12, pp. 305–312, Dec. 2017, doi: <https://doi.org/10.26438/ijcse/v5i12.305312>.
- [5] S. Salloum, R. Dautov, X. Chen, P. X. Peng, and J. Z. Huang, "Big data analytics on Apache Spark," *International Journal of Data Science and Analytics*, vol. 1, no. 3–4, pp. 145–164, Oct. 2016, doi: <https://doi.org/10.1007/s41060-016-0027-9>.
- [6] S. Ait Errami, H. Hajji, K. Ait El Kadi, and H. Badir, "Spatial big data architecture: From Data Warehouses and Data Lakes to the LakeHouse," *Journal of Parallel and Distributed Computing*, vol. 176, pp. 70–79, Jun. 2023, doi: <https://doi.org/10.1016/j.jpdc.2023.02.007>.
- [7] "What is Medallion Architecture?," *Databricks*, 2022. [Online]. Available: <https://www.databricks.com/blog/what-is-medallion-architecture>
- [8] X. Meng *et al.*, "MLlib: Machine Learning in Apache Spark," *arXiv (Cornell University)*, Jan. 2016, doi: <https://doi.org/10.48550/arxiv.1505.06807>.
- [9] L. Eboli, C. Forciniti, and G. Mazzulla, "Factors influencing accident severity: an analysis by road accident type," *Transportation Research Procedia*, vol. 47, pp. 449–456, 2020, doi: <https://doi.org/10.1016/j.trpro.2020.03.120>.
- [10] R. Yu and M. Abdel-Aty, "Analyzing crash injury severity for a mountainous freeway incorporating real-time traffic and weather data," *Safety Science*, vol. 63, pp. 50–56, Mar. 2014, doi: <https://doi.org/10.1016/j.ssci.2013.10.012>.
- [11] L. Qiu and W. A. Nixon, "Effects of Adverse Weather on Traffic Crashes," *Transportation Research Record: Journal of the Transportation Research Board*, vol. 2055, no. 1, pp. 139–146, Jan. 2008, doi: <https://doi.org/10.3141/2055-16>.
- [12] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001. doi: 10.1023/A:1010933404324

APPENDIX

A. Project Timeline

Task / Activity Name	18/6	19/6	20/6	21/6	22/6	23/6	24/6	25/6	26/6	27/6	28/6	29/6	30/6	2/7	3/7	4/7	5/7	6/7
Project instruction review																		
Identify Dataset Sources (Kaggle, API, Census)																		
Introduction																		
Literature review																		
Draft Proposal Submission																		
Proposal Presentation																		
Bronze Layer: Raw Data Ingestion & Extraction																		
Silver Layer: Cleaning, Handling Nulls, Formatting																		
Gold Layer: Broadcast Joins & Feature Engineering																		
Exploratory Data Analysis (EDA) in PySpark																		
Model Training: Random Forest Classifier																		
Model Evaluation																		
Power BI Dashboard Design																		
Methodology																		
System Design																		
Conclusion																		
Final Reporting & Documentation																		

B. Generative AI (GAI) Activity Log

Date	Project Task	GAI Used	Purpose / Activity	Example Prompt Used
18/06/2026	Project planning	ChatGPT	Planned project workflow and methodology	"Suggest a methodology for a Spark-based traffic accident analytics project."
18/06/2026	Data source identification	ChatGPT	Compared suitable datasets and APIs	"Recommend open datasets and APIs for traffic accident analytics."
19/06/2026	Literature review	ChatGPT	Summarized related studies and searched references	"Summarize Medallion Architecture and Apache Spark with APA references."
20/06/2026	Bronze Layer implementation	ChatGPT	Explained data ingestion workflow and reviewed code	"Explain my Bronze Layer implementation based on PySpark code."
21/06/2026	Silver Layer processing	ChatGPT	Summarized data cleaning and preprocessing steps	"Write Data Processing and Transformation section based on my Spark code."
22/06/2026	Gold Layer feature engineering	ChatGPT	Explained feature engineering and dataset integration	"Describe feature engineering activities for the Gold layer."
22/06/2026	Exploratory Data Analysis	ChatGPT	Helped interpret Spark analysis results	"Explain the EDA findings from my accident dataset."

23/06/2026	Random Forest model	ChatGPT	Reviewed machine learning implementation and evaluation metrics	"Explain Random Forest implementation using Spark MLlib."
24/06/2026	Power BI dashboard	ChatGPT	Improved dashboard descriptions and KPI explanations	"Write an academic explanation of my Power BI dashboard."
25/06/2026	Methodology chapter	ChatGPT	Reviewed report structure and improved academic writing	"Rewrite this methodology section in academic English."
26/06/2026	System Design	ChatGPT	Generated UML diagrams and architecture descriptions	"Generate PlantUML code for the use case diagram."
27/06/2026	System Architecture	ChatGPT	Designed ETL architecture diagram and explanation	"Create a simple Medallion architecture diagram."
28/06/2026	Database Design	ChatGPT	Reviewed ERD and star schema documentation	"Explain the ERD and multidimensional database design."
29/06/2026	Testing	ChatGPT	Drafted Alpha Testing table and testing explanation	"Write Alpha Testing based on my dashboard functions."
30/06– 06/07/2026	Final report editing	ChatGPT	Grammar checking, paraphrasing, formatting, and proofreading	"Improve this paragraph while keeping an academic writing style."